CHAPTER 7

The `'static` Lifetime

Sooner or later the compiler tells you that something "must outlive `'static`". It happens when you hand a closure to `thread::spawn`, when you push a boxed callback into a registry, when you return a `Box<dyn Error>`, or when a generic function you wrote calls one of those. You never wrote `'static` anywhere, and the value in question lives for a few microseconds. The quickest way to make the message go away is to write `&'static` on a parameter, and that is usually the wrong fix: it moves the error to every caller.

The confusion has a simple cause. `'static` appears in two different positions, and they say two different things. `&'static T` is a *reference* that stays valid until the program ends. `T: 'static` is a *bound* on a type that says the type holds no borrow that can expire. A `String` created and dropped inside one function satisfies the second and has nothing to do with the first. This chapter separates the two meanings, shows where each one comes from, explains why some APIs insist on `'static`, and ends with a ladder of fixes for when they do.

In this chapter you will learn:

- the difference between `&'static T`, a reference to data that stays valid until the program ends, and `T: 'static`, a bound that says a type holds no borrow that can expire;
- where `&'static` references come from: literals, `static` and `const` items, constant promotion, leaking, and lazily initialized globals (`OnceLock`, `LazyLock`);
- why `T: 'static` does not mean "lives forever", and the one question that tells you whether a type satisfies it;
- why `thread::spawn`, `Any`, stored callbacks and generic code ask for `'static`, and how to read E0373, E0521, E0597 and E0310 when they do;
- the hidden `'static` in `Box<dyn Trait>`;
- the common myths, and a decision ladder for "the compiler wants `'static`": own, scope, mix, leak.

7.1 Two meanings of `'static`

Chapter 4 placed lifetimes in an order: `'a: 'b` ("`'a` outlives `'b`") when the region `'a` contains the region `'b`. `'static` is the largest region of all. It covers the whole run of the program, so `'static: 'a` holds for every lifetime `'a`. That is the only `'static` there is. What changes between the two uses is what the lifetime is attached to:

You write	What it is	What it says
`&'static T`	a reference type	the referent stays valid until the program ends
`T: 'static`	a bound on a type	every borrow inside `T` is valid until the program ends

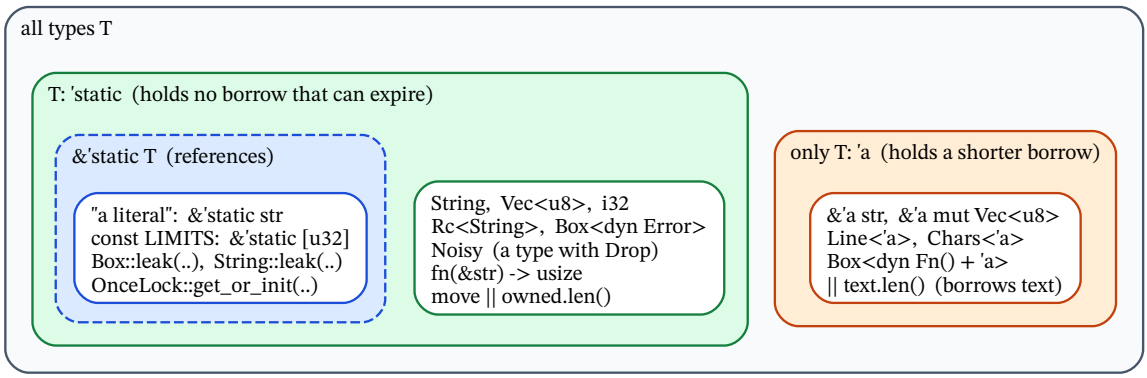
The first is a type: a pointer with a guarantee about its target. A string literal has type `&'static str`, because its bytes are part of the executable. And since `'static` outlives everything, a `&'static T` can be used wherever a `&'a T` is expected, for any `'a` (that is covariance, Chapter 4).

The second is a property of a type, written with the outlives notation of Chapter 1. `T: 'a` means "every lifetime that appears in `T` outlives `'a`", or more simply "every borrow that `T` holds is valid for at least `'a`". With `'static` in place of `'a`, it says that every borrow inside `T` is valid until the program ends. Concretely:

- A type that contains no lifetime at all, such as `String`, `Vec<u8>`, `i32` or `Box<[f64]>`, satisfies `T: 'static` unconditionally. It holds no borrow, so it holds no borrow that can expire.
- A reference type `&'a U` satisfies it only if `'a` is `'static` (and `U` does too). A borrow of a local variable never does.
- A struct with a lifetime parameter, such as `Line<'a>`, satisfies it only if `'a` is `'static`.
- A generic type such as `Vec<T>` or `Rc<T>` satisfies it exactly when `T` does.

So `T: 'static` is not about how long a *value* lives. It is about what the *type* is allowed to contain. A good way to read it is "`T` could, if someone wanted to, be kept until the end of the program without any of its borrows dangling". Whether anyone actually keeps it that long is a different question.

Drawn as sets of types, the two meanings look like this:



The types that satisfy `T: 'static` hold no borrow that can expire; the `&'static T` reference types are a small part of them, and types that hold a shorter borrow satisfy only `T: 'a`.

The diagram makes two points. First, every `&'static T` (with `T: 'static`) satisfies the bound, but most types that satisfy the bound are not references at all: they are owning types. Second, the types outside the green set are not defective. `&'a str` or `Line<'a>` are perfectly good types; they hold a borrow, so they can be used only while that borrow is valid, and an API that needs to keep a value for an unknown amount of time cannot accept them.

7.2 Where `&'static` references come from

A `&'static T` must point to something that is never freed, or at least not before the program ends. There are five sources of such references: literals, `static` and `const` items, constant promotion, leaking, and lazily initialized globals. The program `code/examples/ch07_static_sources.rs` uses the first four.

Literals, `static` items and `const` items

```
// The type of a string literal is `&'static str`: the bytes are part of the binary.
static BANNER: &str = "lifetimes"; // elided: `&'static str`
const LIMITS: &[u32] = &[1, 2, 3]; // elided: `&'static [u32]`

// A `static` item lives at one address for the whole run, so `&GREETING` is
// `&'static String`, even though `String` has a destructor (it never runs).
static GREETING: String = String::new();
```

A string literal is stored in the read-only data of the executable, so it exists before `main` starts and after it returns: its type is `&'static str`. A `static` item is a single value at a fixed address for the whole run of the program, and it is never dropped, so `&GREETING` is a `&'static String` even though `String` has a destructor (the destructor simply never runs). A `const` item is different: it has no address of its own, and its value is copied into every place that uses it. A `const` of reference type, such as `LIMITS`, holds a reference into static memory, so it is `&'static` too.

Notice that `BANNER` and `LIMITS` are written `&str` and `&[u32]` without a lifetime. In the type of a `static` or `const` item, an elided lifetime means `'static` (Chapter 3 lists this among the elision rules). It is the only sensible choice: a global has no shorter region to borrow from.

Constant promotion

The second source is less visible. When you write `&` in front of a *constant expression* inside a function, the compiler can store the value in static memory instead of in a temporary on the stack, and give you a `&'static` reference to it. This is called *constant promotion*:

```
fn promoted_number() -> &'static i32 {
    &42 // a constant expression: promoted to an anonymous static
}

fn promoted_table() -> &'static [&'static str] {
    &["fn", "let", "match"] // an array of literals is a constant too
}
```

Without promotion, `&42` would be a reference to a temporary that dies at the end of the function, and returning it would be an error. With promotion, it is a reference to an anonymous static that holds `42`. The same goes for an array of literals.

Promotion has limits, though. The value must be a constant, and it must not have a destructor or interior mutability: a promoted value is shared by every call of the function, so it must be something that can neither be dropped nor changed. The program `code/compile_fail/ch07_not_promoted.rs` tries both:

```
pub fn empty_name() -> &'static String {
    &String::new() // `String` has a destructor: not promoted
}

pub fn counter() -> &'static Cell<u32> {
    &Cell::new(0) // interior mutability: not promoted
}
```

The compiler rejects both functions with the same error:

```
error[E0515]: cannot return reference to temporary value
 --> compile_fail/ch07_not_promoted.rs:9:5
  |
9 |     &String::new() // `String` has a destructor: not promoted
  |     ^-------------
  |     ||
  |     |temporary value created here
  |     returns a reference to data owned by the current function

error[E0515]: cannot return reference to temporary value
  --> compile_fail/ch07_not_promoted.rs:13:5
   |
13 |     &Cell::new(0) // interior mutability: not promoted
   |     ^------------
   |     ||
   |     |temporary value created here
   |     returns a reference to data owned by the current function
```

Read the first diagnostic label by label. "Temporary value created here" points at `String::new()`: the value is an ordinary temporary, not a promoted constant. "Returns a reference to data owned by the current function" points at the `&`: the reference would outlive the temporary, which is dropped when the function returns. Error E0515 is the general "returning a reference to a local" error, which Chapter 5 covers in detail; here it tells you that promotion did not happen. If you really need a `&'static String`, declare a `static` item, as `GREETING` does above. More often, the right answer is to return an owned `String`.

Leaking

The third source turns heap data into static data, by giving up on ever freeing it. `Box::leak`, `String::leak` and `Vec::leak` consume an owning value and return a reference to its contents:

```
fn leaked(name: String) -> &'static str {
    // The heap buffer is never freed: the reference stays valid until exit.
    name.leak()
}
```

The signature of `String::leak` in the standard library is `pub fn leak<'a>(self) -> &'a mut str` (stable since Rust 1.72; `Box::leak` has the same shape). The lifetime `'a` appears only in the output, so the caller picks it, and `'static` is a valid choice: nothing will ever free the buffer, so the reference can be valid for as long as anyone wants. Note that the result is a *mutable* reference: `&'static mut str` is a perfectly good type.

Leaking is safe (Chapter 13 explains why Rust treats it as safe), but it is a memory leak by definition. It is the right tool for data that must live until the program exits anyway, such as a configuration loaded once at startup or a table of interned names. It is a bug when it happens once per request or once per loop iteration, because the memory use then grows without bound.

Here are the first four sources at work:

```
let literal: &'static str = "a literal";
let empty: &'static String = &GREETING;
let label: &'static str = leaked(format!("{}-{}", "worker", 7));
println!("literal:  {literal}");
println!("static:   {BANNER} and {empty:?}");
println!("const:    {LIMITS:?}");
println!("promoted: {} and {:?}", promoted_number(), promoted_table());
println!("leaked:   {label}");
```

```
literal:  a literal
static:   lifetimes and ""
const:    [1, 2, 3]
promoted: 42 and ["fn", "let", "match"]
leaked:   worker-7
```

Lazily initialized globals: `OnceLock` and `LazyLock`

A `static` item must be initialized with a constant expression, which rules out anything that allocates, reads a file or parses text. The standard library has two types that fill this gap: `OnceLock<T>` (stable since Rust 1.70) is a cell that can be written once, and `LazyLock<T>` (since Rust 1.80) is a value computed by a closure the first time it is used. Both are safe to use from several threads. Put one in a `static`, and you get `&'static` references to data built at run time. The program is `code/examples/ch07_globals.rs`:

```
static CONFIG: OnceLock<Config> = OnceLock::new();

/// Initializes the configuration once; later calls return the same value.
fn config() -> &'static Config {
    CONFIG.get_or_init(|| {
        println!("(loading config)");
        Config {
            name: String::from("indexer"),
            workers: 4,
        }
    })
}
```

The signature of `OnceLock::get_or_init` is `pub fn get_or_init<F>(&self, f: F) -> &T`, with the output lifetime elided: the result borrows from `&self` (elision rule 3, Chapter 3). Since `CONFIG` is a `static`, `&CONFIG` is `&'static OnceLock<Config>`, so the result is a `&'static Config`. No leak and no `unsafe` are involved; the global was always going to live until the end of the program.

`LazyLock` puts the initializer in the declaration. It dereferences to the value, so you can call methods on it directly:

```
static KEYWORDS: LazyLock<BTreeMap<&'static str, usize>> = LazyLock::new(|| {
    println!("(building keyword table)");
    ["fn", "let", "match", "static"]
        .into_iter()
        .enumerate()
        .map(|(id, word)| (word, id))
        .collect()
});

fn keyword_id(word: &str) -> Option<usize> {
    KEYWORDS.get(word).copied()
}
```

```
let first: &'static Config = config();
let second: &'static Config = config(); // no second "(loading config)"
assert!(std::ptr::eq(first, second));
println!("{} with {} workers", first.name, second.workers);

println!("static -> {:?}", keyword_id("static"));
println!("struct -> {:?}", keyword_id("struct"));
```

```
(loading config)
indexer with 4 workers
(building keyword table)
static -> Some(3)
struct -> None
```

Each initializer runs once, the first time the global is used, and both calls to `config()` return the same reference (the `assert!` checks the address). Use `OnceLock` when the code that initializes the global is somewhere else (for example `main` sets it from the command line); use `LazyLock` when the initializer is self-contained. Chapter 12 shows `OnceCell`, the single-threaded version, whose `get_or_init` returns a reference tied to the cell rather than a `&'static` one.

7.3 `T: 'static` does not mean "lives forever"

Now the bound. The most common misreading is that `T: 'static` means "the value lives until the end of the program". The program `code/examples/ch07_static_type_dropped.rs` shows that it does not. `Noisy` is a type whose only field is a `&'static str`, so it satisfies `T: 'static`. It prints a line when it is dropped:

```
/// Prints a line when it is dropped, so we can see when that happens.
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("  drop {}", self.0);
    }
}

/// Accepts any value whose type holds no borrow that can expire.
fn requires_static<T: 'static>(_value: T) {
    println!("  inside requires_static");
} // `_value` is dropped here, like any parameter
```

```
println!("call 1");
requires_static(Noisy("first")); // `Noisy: 'static`, and it dies right away
println!("call 2");
let local = String::from("owned by main");
requires_static(local); // a local `String` is fine: it is moved in
println!("call 3");
requires_static(7_u8);
requires_static("a literal"); // `&'static str: 'static`
println!("done");
```

```
call 1
  inside requires_static
  drop first
call 2
  inside requires_static
call 3
  inside requires_static
```

```
  inside requires_static
done
```

The `Noisy("first")` value is created in `main`, moved into `requires_static`, and dropped at the end of that call, before "call 2" is printed. Its type is `'static`; the value lived for a few lines. The same is true of the `String` created by `main`: it is a local, and it is moved into the function and dropped there. `7_u8` and a string literal are accepted too.

What fails the bound is a *borrow* of a local variable. In `code/compile_fail/ch07_static_bound_borrow.rs`, the caller passes `&local` instead of `local`:

```
pub fn caller() {
    let local = String::from("owned by caller");
    requires_static(&local); // `&'a String: 'static` only if `'a` is `'static`
}
```

The compiler rejects it:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch07_static_bound_borrow.rs:10:21
   |
 9 |     let local = String::from("owned by caller");
   |         ----- binding `local` declared here
10 |     requires_static(&local); // `&'a String: 'static` only if `'a` is `'static`
   |     ----------------^^^^^^_
   |     |               |
   |     |               borrowed value does not live long enough
   |     argument requires that `local` is borrowed for `'static`
11 | }
   | - `local` dropped here while still borrowed
   |
note: requirement that the value outlives `'static` introduced here
  --> compile_fail/ch07_static_bound_borrow.rs:5:23
   |
 5 | fn requires_static<T: 'static>(_value: T) {}
   |                       ^^^^^^^
```

The labels tell the whole story. `T` is inferred as `&'a String` for some region `'a`. The bound `T: 'static` requires `'a` to be `'static` ("argument requires that `local` is borrowed for `'static`"), but `local` is dropped at the closing brace ("`local` dropped here while still borrowed"), so no borrow of it can last that long ("borrowed value does not live long enough"). The note at the bottom is the most useful line: it points at the place where the requirement comes from, the `T: 'static` bound of `requires_static`. When you meet a "borrowed for `'static`" error in real code, look for that note: it tells you which API asked for `'static`.

The test: does it hold a borrow that can expire?

So the practical question is not "how long does the value live?" but "does the type hold a borrow that can expire?". Here is the answer for common types:

Type	`T: 'static`?	Why
`String`, `Vec<String>`, `i32`	yes	owns all its data
`&'static str`	yes	the only borrow inside is `'static`
`Rc<String>`, `Arc<String>`	yes	`Rc<T>: 'static` exactly when `T: 'static`
`Box<dyn Error>`	yes	means `Box<dyn Error + 'static>`
`fn(&str) -> usize`	yes	a function pointer holds no data
a `move` closure owning a `String`	yes	captures by value
`&'a str` (borrowing a local)	no	holds a short borrow
`Line<'a>` (borrowing a local)	no	its lifetime parameter is not `'static`
a closure that uses a local by reference	no	captures a borrow

Every "yes" row is checked by `code/examples/ch07_static_table.rs`, which passes one value of each type to a function with a `T: 'static` bound. To make the run visible, the function prints the name of the type it received:

```
is_static(&String::from("owned"));
is_static(&vec![String::from("a"), String::from("b")]);
is_static(&42_i32);
is_static(&"a literal"); // `&'static str`
is_static(&Rc::new(String::from("shared"))); // `Rc<T>: 'static` iff `T: 'static`
let error: Box<dyn Error> = "boom".into(); // `Box<dyn Error + 'static>`
is_static(&error);
is_static(&(word_count as fn(&str) -> usize)); // a function pointer
let owned = String::from("captured by move");
is_static(&move || owned.len()); // a `move` closure that owns a `String`
```

```
alloc::string::String
alloc::vec::Vec<alloc::string::String>
i32
&str
alloc::rc::Rc<alloc::string::String>
alloc::boxed::Box<dyn core::error::Error>
fn(&'_ str) -> usize
ch07_static_table::main::{{closure}}
```

The output contains a small lesson of its own. `std::any::type_name` prints `&str` for a `&'static str` and `Box<dyn core::error::Error>` without the `+ 'static`: lifetimes are erased before the program runs (Chapter 1), so the running program cannot even name them. Keep this in mind; it explains the `Any` rule below. (The exact text of `type_name` is not guaranteed and is meant for diagnostics only.)

Every "no" row is in `code/compile_fail/ch07_static_table_no.rs`:

```
pub fn caller() {
    let text = String::from("local text");
    is_static(&text.as_str()); // `&'a str` with a local `'a`
    is_static(&Line(&text)); // `Line<'a>` with a local `'a`
    is_static(&|| text.len()); // a closure that borrows `text`
}
```

The compiler rejects all three lines: the first two with the E0597 we just read, and the closure with E0373, which we will read in the next section, because it is the error you get from `thread::spawn`. The full diagnostics are in `code/compile_fail/ch07_static_table_no.stderr`.

7.4 Why APIs ask for 'static

When you call a function, the signature is the contract (Chapter 1). If the function only uses your value during the call, it can accept borrows of any lifetime: the borrow outlives the call, and that is all it needs. An API asks for `T: 'static` when it keeps your value for a span of time that it cannot describe with a lifetime parameter: a thread that may still run after your function has returned, a value whose type has been erased, a callback stored in a registry that lives as long as the program. In each case the callee has no shorter region to name, so it asks for the only one that is guaranteed to be long enough.

Threads: `spawn` and `thread::scope`

Here is the signature of `thread::spawn` in Rust 1.99, without its attributes and documentation:

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T,
    F: Send + 'static,
    T: Send + 'static,
```

The closure `F` and its result `T` must both be `Send` (they cross to another thread) and `'static` (they may stay there for an unknown time). Why "unknown"? A `JoinHandle` can be dropped without joining; the thread then keeps running, detached, possibly after the function that spawned it has returned and its locals have been freed. The signature of `spawn` cannot know whether you will join, so it must assume you will not.

The program `code/compile_fail/ch07_spawn_borrow.rs` spawns a closure that uses a local vector:

```
pub fn caller() {
    let words = vec![String::from("alpha"), String::from("beta")];
    let handle = thread::spawn(|| words.len()); // borrows `words`
    println!("{}", handle.join().unwrap());
}
```

The compiler rejects it:

```
error[E0373]: closure may outlive the current function, but it borrows `words`, which is
owned by the current function
  --> compile_fail/ch07_spawn_borrow.rs:10:32
   |
10 |     let handle = thread::spawn(|| words.len()); // borrows `words`
   |                                ^^ ----- `words` is borrowed here
   |                                |
   |                                may outlive borrowed value `words`
   |
note: function requires argument type to outlive `'static`
  --> compile_fail/ch07_spawn_borrow.rs:10:18
   |
10 |     let handle = thread::spawn(|| words.len()); // borrows `words`
   |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: to force the closure to take ownership of `words` (and any other referenced
variables), use the `move` keyword
   |
10 |     let handle = thread::spawn(move || words.len()); // borrows `words`
   |                                ++++
```

Error E0373 is reported for a closure (or an `async` block) that captures a local variable by reference and must outlive the function. The closure uses `words` only through a `&self` method (`len`), so it

captures `words` by reference, and its type holds a borrow of a local: it is not `'static`. The first label, "may outlive borrowed value `words`", says why that is a problem; the note "function requires argument type to outlive `'static`" says who demands it (`spawn`). The help is right this time: with `move`, the closure takes ownership of the vector, its type holds no borrow, and the program compiles. Notice also that the `join` on the next line changes nothing. The compiler checks the call to `spawn` against the signature of `spawn`, not against what you do afterward.

The help is not always right, because `move` only moves what the closure captures, and sometimes that is already a reference. The program `code/compile_fail/ch07_spawn_param.rs` receives the data by reference:

```
pub fn count_in_background(words: &[String]) -> usize {
    let handle = thread::spawn(move || words.len()); // moves the *reference* in
    handle.join().unwrap()
}
```

The closure is `move`, so it owns its capture, but its capture is `words`, a `&[String]`. Moving a reference moves the borrow, not the data. The compiler reports a different error:

```
error[E0521]: borrowed data escapes outside of function
 --> compile_fail/ch07_spawn_param.rs:9:18
  |
8 | pub fn count_in_background(words: &[String]) -> usize {
  |                            -----  - let's call the lifetime of this reference `'1`
  |                            |
  |                            `words` is a reference that is only valid in the function
body
9 |     let handle = thread::spawn(move || words.len()); // moves the *reference* in
  |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |                  |
  |                  `words` escapes the function body here
  |                  argument requires that `'1` must outlive `'static`
```

E0521, "borrowed data escapes outside of function", appears when a borrow that came in through a parameter flows into something that must outlive the function. Read it label by label:

- "`words` is a reference that is only valid in the function body": from the function's point of view, the lifetime of `words` is chosen by the caller and covers at least the call, but nothing more is known about it.
- "let's call the lifetime of this reference `'1`": the parameter's lifetime is elided, so it has no name; rustc invents one (`'1`) to talk about it.
- "`words` escapes the function body here": the call to `spawn`, which wants to keep the closure, and therefore the borrow, for an unknown time.
- "argument requires that `'1` must outlive `'static`": the requirement that cannot be proved. The caller might pass a borrow of a local of its own.

There are two honest fixes. Either the thread owns the data (take a `Vec<String>` or an `Arc<[String]>` parameter, or clone inside the function), or the thread does not outlive the function, which is exactly what scoped threads guarantee. The program `code/examples/ch07_threads.rs` shows the owning fixes first:

```
// 1. Own: the closure owns a clone, so its type is `'static`.
let owned = words.clone();
let handle = thread::spawn(move || owned.join(" "));
println!("moved:  {}", handle.join().unwrap());
```

```
// 2. Share: `Arc<Vec<String>>` owns its data, so it is `'static` too.
let shared = Arc::new(words.clone());
let handles: Vec<_> = (0..shared.len())
    .map(|i| {
        let shared = Arc::clone(&shared);
        thread::spawn(move || shared[i].len())
    })
    .collect();
let lengths: Vec<usize> = handles.into_iter().map(|h| h.join().unwrap()).collect();
println!("shared: {lengths:?}");
```

The first closure owns a clone of the vector. The others each own an `Arc<Vec<String>>`, a shared handle that keeps the vector alive as long as any thread holds one: `Arc<T>` owns its contents, so it is `'static` when `T` is. Cloning an `Arc` copies a pointer and increments a counter; it does not copy the strings.

The third way borrows. `thread::scope` (stable since Rust 1.63) calls your closure with a `Scope` value, and every thread spawned through that `Scope` is joined before `thread::scope` returns. Since the threads cannot outlive the call, they may borrow anything that outlives the call:

```
// 3. Borrow: every thread spawned in the scope is joined before `scope`
//    returns, so the closures may borrow `words`.
let (total, longest) = thread::scope(|s| {
    let total = s.spawn(|| words.iter().map(|w| w.len()).sum::<usize>());
    let longest = s.spawn(|| words.iter().max_by_key(|w| w.len()).unwrap());
    (total.join().unwrap(), longest.join().unwrap())
});
println!("scoped: total {total}, longest {longest}"); // `longest: &String`
println!("still mine: {words:?}");
```

```
moved:  alpha beta gamma
shared: [5, 4, 5]
scoped: total 14, longest gamma
still mine: ["alpha", "beta", "gamma"]
```

The signature says the same thing in lifetimes. The bounds of `Scope::spawn` are `F: FnOnce() -> T + Send + 'scope` and `T: Send + 'scope`, where the lifetime `'scope` is the region of the call to `thread::scope`. The data the closures borrow lives in a longer region (the standard library calls it `'env`), so the borrows satisfy `'scope`. A scoped thread can even *return* a borrow: `longest` is a `&String` pointing into `words`, and it is still usable after the scope ends. And `words` is still owned by `main` at the end. Why `thread::scope` takes a closure instead of returning a guard that joins on drop is a story about leaks, told in Chapter 13; why async runtimes cannot easily offer the same thing is told in Chapter 11.

`Any` and `TypeId`

`std::any::Any` lets you erase the type of a value and recover it later by checking at run time. Its definition starts like this:

```
pub trait Any: 'static {
    fn type_id(&self) -> TypeId;
```

```
}

impl<T: 'static + ?Sized> Any for T { /* ... */ }
```

and `TypeId::of` is declared as `pub const fn of<T: ?Sized + 'static>() -> TypeId`. `'static` everywhere. The program `code/examples/ch07_any.rs` uses them to build a type map, a collection that holds at most one value per type, like the "extensions" that web frameworks attach to a request:

```
/// At most one value per type, like the "extensions" of an HTTP request.
#[derive(Default)]
struct Extensions {
    values: HashMap<TypeId, Box<dyn Any>>, // `Box<dyn Any + 'static>`
}

impl Extensions {
    fn insert<T: 'static>(&mut self, value: T) {
        self.values.insert(TypeId::of::<T>(), Box::new(value));
    }

    fn get<T: 'static>(&self) -> Option<&T> {
        self.values.get(&TypeId::of::<T>())?.downcast_ref::<T>()
    }
}
```

```
let mut extensions = Extensions::default();
extensions.insert(RequestId(42));
extensions.insert(UserName(String::from("ada")));
extensions.insert("a literal"); // `&'static str` is a `'static` type

let id: Option<&RequestId> = extensions.get::<RequestId>();
let user: Option<&UserName> = extensions.get::<UserName>();
println!("request id: {:?}", id.map(|id| id.0));
println!("user name:  {:?}", user.map(|user| user.0.as_str()));
println!("literal:    {:?}", extensions.get::<&'static str>());
println!("String:     {:?}", extensions.get::<String>()); // never inserted
```

```
request id: Some(42)
user name:  Some("ada")
literal:    Some("a literal")
String:     None
```

`insert` stores each value under the `TypeId` of its type; `get` looks up the `TypeId` of the requested type and asks `downcast_ref` to turn the `&dyn Any` back into a `&T`. `downcast_ref` succeeds only if the stored value's `TypeId` is the `TypeId` of `T`. Removing `T: 'static` from `insert` or `get` would be E0310, which we meet below.

Why must the types be `'static`? Because a `TypeId` is computed by the compiler after lifetimes have been erased, and the running program has no lifetimes to compare. `Token<'a>` and `Token<'static>` would get the same `TypeId`. If `Any` accepted borrowed types, downcasting could turn a short borrow into a `'static` one. In this pseudo-code, which does *not* compile, `forge` would hand out a reference to data that is about to be freed:

```
// Not real Rust: imagine `Any` without its `'static` bound.
fn forge<'a>(text: &'a str) -> &'static str {
    let erased: Box<dyn Any + 'a> = Box::new(text); // TypeId of `&str`...
    *erased.downcast::<&'static str>().unwrap() // ...equals TypeId of `&'static str`
}
```

The 'static bound on Any rules this out: only types whose lifetimes are all 'static can be erased, and for those, forgetting the lifetimes loses nothing.

So a type that borrows cannot become a dyn Any. Here is an attempt, in code/compile_fail/ch07_any_borrowed.rs:

```
/// A token that borrows its text from the source it was read from.
pub struct Token<'src> {
    pub text: &'src str,
}

pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any> {
    Box::new(token)
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch07_any_borrowed.rs:14:5
   |
13 | pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any> {
   |              ---- lifetime `'src` defined here
14 |     Box::new(token)
   |     ^^^^^^^^^^^^^^^ coercion requires that `'src` must outlive `'static`
   |
help: to declare that the trait object captures data from argument `token`, you can add an
explicit `'src` lifetime bound
   |
13 | pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any + 'src> {
   |                                                       ++++++
```

This error has no code; its message is "lifetime may not live long enough". The label "coercion requires that 'src must outlive 'static" refers to the *unsizing coercion* from Box<Token<'src>> to Box<dyn Any> (Chapter 8), and the requirement comes from two places: Box<dyn Any> means Box<dyn Any + 'static>, and Any itself is only implemented for 'static types.

The help suggests writing Box<dyn Any + 'src>. That is a generic suggestion, made for every trait object, and here it does not fix anything. The program code/compile_fail/ch07_any_help_moves_error.rs follows it:

```
pub fn erase<'src>(token: Token<'src>) -> Box<dyn Any + 'src> {
    Box::new(token) // compiles: the signature now implies `'src: 'static`
}

pub fn caller() {
    let source = String::from("let x = 1;");
    let erased = erase(Token { text: &source[..3] });
    println!("{}", erased.is::<Token<'static>>());
}
```

Now erase compiles, but the caller does not:

```
error[E0597]: `source` does not live long enough
  --> compile_fail/ch07_any_help_moves_error.rs:19:39
   |
18 |     let source = String::from("let x = 1;");
   |         ------ binding `source` declared here
19 |     let erased = erase(Token { text: &source[..3] });
   |                  ----------------------^^^^^^--------
   |                  |                     |
   |                  |                     borrowed value does not live long enough
   |                  argument requires that `source` is borrowed for `'static`
20 |     println!("{}", erased.is::<Token<'static>>());
```

```
21 | }
   | - `source` dropped here while still borrowed
```

Because `Any: 'static`, the type `dyn Any + 'src` is only well-formed when `'src: 'static`. The compiler assumes that the types in a signature are well-formed, so the signature of `erase` now carries an *implied bound* `'src: 'static` (implied bounds were introduced in Chapter 1). `erase` compiles, but it accepts only tokens that borrow `'static` data, and the error reappears at every call site that has a real borrow. The fix is to erase only owned data: convert the token into an owned version first (Chapter 14 shows a borrowed token type with an owned twin), or do not use `Any` for borrowed data at all; an `enum` of the possible types, or a trait with the methods you need, works with any lifetime.

Stored callbacks and the hidden `'static` of `Box<dyn Trait>`

A registry of callbacks keeps closures for as long as the registry lives. Here is one from `code/examples/ch07_callbacks.rs`:

```
#[derive(Default)]
struct Registry {
    handlers: Vec<Box<dyn Fn(&str) -> String>>, // `+ 'static` is implied
}

impl Registry {
    fn register(&mut self, handler: Box<dyn Fn(&str) -> String>) {
        self.handlers.push(handler);
    }

    fn run(&self, line: &str) {
        for handler in &self.handlers {
            println!("{}", handler(line));
        }
    }
}
```

The field type is written `Box<dyn Fn(&str) -> String>`, with no lifetime. A trait object always has a lifetime bound, the `+ 'a` in `dyn Trait + 'a`, which says how long the hidden value may borrow. When you leave it out, the compiler picks a default, and inside a `Box` the default is `'static`. So the field is really `Box<dyn Fn(&str) -> String + 'static>`, and only closures that hold no expiring borrow fit. The program registers two that do:

```
let mut registry = Registry::default();
let prefix = String::from(">> ");
registry.register(Box::new(move |line| format!("{prefix}{line}"))); // owns `prefix`
registry.register(Box::new(|line| line.to_uppercase())); // captures nothing
registry.run("hello");
```

```
>> hello
HELLO
```

The first closure is `move`, so it owns `prefix`; the second captures nothing. Remove the `move`, and the closure borrows `prefix`. The program `code/compile_fail/ch07_callback_borrow.rs` does that:

```
pub struct Registry {
    handlers: Vec<Box<dyn Fn(&str) -> String>>,
}

impl Registry {
```

```
    pub fn register(&mut self, handler: Box<dyn Fn(&str) -> String>) {
        self.handlers.push(handler);
    }
}

pub fn caller(registry: &mut Registry) {
    let prefix = String::from(">> ");
    registry.register(Box::new(|line| format!("{prefix}{line}"))); // borrows `prefix`
}
```

```
error[E0597]: `prefix` does not live long enough
  --> compile_fail/ch07_callback_borrow.rs:18:49
   |
17 |     let prefix = String::from(">> ");
   |         ------ binding `prefix` declared here
18 |     registry.register(Box::new(|line| format!("{prefix}{line}"))); // borrows `prefix`
   |                       --------------------------^^^^^^----------
   |                       |        |                |
   |                       |        |                borrowed value does not live long
enough
   |                       |        value captured here
   |                       coercion requires that `prefix` is borrowed for `'static`
19 | }
   | - `prefix` dropped here while still borrowed
   |
   = note: due to object lifetime defaults, `Box<dyn for<'a> Fn(&'a str) -> String>`
actually means `Box<(dyn for<'a> Fn(&'a str) -> String + 'static)>`
```

"Value captured here" points at the closure, which captures `prefix` by reference. "Coercion requires that `prefix` is borrowed for `'static`" points at `Box::new(...)`, where the boxed closure becomes a `Box<dyn Fn ...>`. And the note spells out the hidden default: `Box<dyn for<'a> Fn(&'a str) -> String>` "actually means" `Box<(dyn for<'a> Fn(&'a str) -> String + 'static)>`. (The `for<'a>` is how the compiler writes the elided lifetime of the `&str` argument; Chapter 9 explains it.)

There are two fixes, and they are design decisions. Either the callbacks own what they use (`move`, as in the working program), which keeps the registry free of lifetime parameters and lets it live anywhere. Or the registry admits that its callbacks borrow: `struct Registry<'a>` with a field of type `Vec<Box<dyn Fn(&str) -> String + 'a>>`, which accepts borrowing closures but can then live only as long as what they borrow. Chapter 8 gives the complete rules for the default bounds of trait objects (they differ for `&'a dyn Trait`, for example), and Chapter 14 builds a registry of the second kind (a `Dispatcher<'a>`).

The same hidden `'static` is behind the trouble with `Box<dyn Error>`: an error type that borrows from its input cannot become a `Box<dyn Error>`. Chapter 8 works through that case.

Generic code: E0310

The `'static` requirements of `spawn`, `Any` and boxed trait objects do not stay inside those APIs. If your generic function passes its `T` to one of them, the requirement must appear in your signature too. The program `code/compile_fail/ch07_generic_spawn.rs` moves a value to another thread to drop it there, a trick sometimes used to keep an expensive destructor off a latency-sensitive thread:

```
pub fn drop_in_background<T: Send>(value: T) -> thread::JoinHandle<()> {
    thread::spawn(move || drop(value))
}
```

```
error[E0310]: the parameter type `T` may not live long enough
 --> compile_fail/ch07_generic_spawn.rs:9:5
  |
9 |     thread::spawn(move || drop(value))
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |     |
  |     the parameter type `T` must be valid for the static lifetime...
  |     ...so that the type `T` will meet its required lifetime bounds
  |
help: consider adding an explicit lifetime bound
  |
8 | pub fn drop_in_background<T: Send + 'static>(value: T) -> thread::JoinHandle<()> {
  |                                   +++++++++
```

E0310 is "the parameter type `T` may not live long enough". The first label says what `spawn` needs ("must be valid for the static lifetime"), the second why ("so that the type `T` will meet its required lifetime bounds": the closure owns a `T`, and the closure must be `'static`). Inside `drop_in_background`, `T` is unknown: the caller could choose `T = &'a String` with a short `'a`. The body is checked against the signature, and the signature promises nothing about `T`'s borrows. The help gives the fix, and this time it is the right one: add the bound to the signature, so that callers are checked against it.

`code/examples/ch07_generic_spawn.rs`:

```rust
fn drop_in_background<T: Send + 'static>(value: T) -> thread::JoinHandle<()> {
    thread::spawn(move || drop(value))
}
```

```rust
let table = Table((0..1000).map(|i| format!("row {i}")).collect());
let handle = drop_in_background(table); // `Table: Send + 'static`
handle.join().unwrap();
println!("joined");
```

```
dropping a table of 1000 rows
joined
```

The bound is part of the contract now. A caller with an owned `Table` is fine; a caller that tries to pass `&table` gets the E0597 we read earlier, at the call site, where the borrow is visible. This is how `'static` propagates through generic code: up the call chain, one signature at a time, until it reaches code that knows the concrete type.

7.5 Myths and anti-patterns

With the two meanings separated, most of the folklore around `'static` falls apart:

Myth or habit	Reality
"`T: 'static` means the value lives for the whole program."	It restricts the type. A `String` local to one function is `'static`, and it is dropped as usual (`ch07_static_type_dropped`).
"`&'static T` and `T: 'static` are the same thing."	The first is a reference to data that lives until the program ends; the second says that a type holds no borrow that can expire.
"`'static` data is in the executable."	Only literals and constant data are. Leaked buffers and `OnceLock` contents live on the heap.
"The compiler wants `'static`, so I'll write `&'static` on my parameter."	That forces every caller to supply a literal or leaked data. Find the API that asked for `'static` and give it owned data.
"Leak it; it's only a small `String`."	Leaking per call is an unbounded memory leak. Leak only data that lives until exit anyway.
"`struct Config { name: &'static str }` is flexible."	It accepts only literals and leaked strings. Use `String`, `Arc<str>`, `Cow<'static, str>` or a lifetime parameter (Chapter 14).

The fourth row deserves a program, because it is the most common wrong fix. Suppose you started with the E0521 above, in a function that starts a logger thread with a prefix of type `&str`. Writing `&'static str` makes the function compile. Here it is, with a caller, in `code/compile_fail/ch07_static_param.rs`:

```
fn start_logger(prefix: &'static str) -> thread::JoinHandle<()> {
    thread::spawn(move || println!("{prefix}: started"))
}

pub fn caller(id: u32) {
    let prefix = format!("worker-{id}");
    start_logger(&prefix).join().unwrap();
}
```

```
error[E0597]: `prefix` does not live long enough
  --> compile_fail/ch07_static_param.rs:14:18
   |
13 |     let prefix = format!("worker-{id}");
   |         ------ binding `prefix` declared here
14 |     start_logger(&prefix).join().unwrap();
   |     -------------^^^^^^^_
   |     |            |
   |     |            borrowed value does not live long enough
   |     argument requires that `prefix` is borrowed for `'static`
15 | }
   | - `prefix` dropped here while still borrowed
```

The error did not go away; it moved from the function to its caller, which builds the prefix at run time and cannot produce a `'static` borrow of it. Every caller with a `String` will meet the same error. The real fix is to ask for ownership, because the thread really does need to own its prefix. Taking `impl Into<String>` keeps the call sites short (`code/examples/ch07_logger.rs`):

```rust
fn start_logger(prefix: impl Into<String>) -> thread::JoinHandle<()> {
    let prefix: String = prefix.into(); // owned, so the closure is `'static`
    thread::spawn(move || println!("{prefix}: started"))
}
```

```rust
start_logger("main").join().unwrap(); // a literal
for id in 1..=2 {
    start_logger(format!("worker-{id}")).join().unwrap(); // a `String`
}
```

```
main: started
worker-1: started
worker-2: started
```

A literal is converted into a `String` (one small allocation), and a `String` is moved in without copying. The signature now says the truth: this function keeps its argument.

7.6 A decision ladder

When the compiler asks for `'static`, first find out who asks: the note "requirement ... introduced here" or "function requires argument type to outlive `'static`" names the API. Then climb this ladder from the top, and stop at the first rung that fits:

1. **Own it.** Give the API data it owns: move the value in, clone it, convert `&str` to `String`, or share ownership with `Arc` (or `Rc` on one thread). This is the answer most of the time, and it is cheap more often than you think: a clone of a configuration value at startup costs nothing that matters.
2. **Scope it.** If the API has a scoped version, use it: `thread::scope` instead of `thread::spawn`, a `Registry<'a>` with `dyn Fn() + 'a` instead of a `'static` one, a function that takes a callback and calls it before returning instead of storing it. Scoped APIs accept borrows because they promise to be done with them before the borrowed data goes away.
3. **Mix.** If most values are literals and a few are built at run time, `Cow<'static, str>` holds either a `&'static str` or a `String`, and is `'static` in both cases (Chapter 14).
4. **Leak or make it global.** `Box::leak`, `String::leak`, `OnceLock` and `LazyLock`, only for data that genuinely lives until the program exits: configuration, interned names, lookup tables.

What is not on the ladder: writing `&'static` in a signature because an error mentioned `'static`, and `unsafe` code that "extends" a lifetime. The first moves the error to the callers; the second turns a compile error into undefined behavior.

7.7 Common confusions

"The error says `local` must be borrowed for `'static`, so `local` should be a `static`." The error says that a borrow of `local` reached an API that needs `'static`. Making the data global is rung 4 of the ladder; usually rung 1 (give the API an owned value) is the fix.

"`move` makes a closure `'static`." `move` makes the closure own its captures. If a capture is itself a reference, the closure owns a reference and still holds a borrow (E0521 in `ch07_spawn_param`).

"**`T: 'static` and `Send` are the same requirement.**" They are independent. `Rc<String>` is `'static` but not `Send`; `&'a i32` is `Send` but not `'static` unless `'a` is `'static`. `thread::spawn` asks for both.

"**A `&'static` reference is always immutable.**" `Box::leak` and `String::leak` return `&'static mut` references; what `'static` promises is validity, not immutability.

"**`T: 'static` excludes references.**" It excludes references that can expire. `&'static str` satisfies it, which is why the type map above accepts a literal.

"**Following the compiler's help always fixes the error.**" The help for a `'static` requirement is often right (add `move`, add `T: 'static`), but for `dyn Any` it only moves the error to the callers, and adding `&'static` to a parameter does the same. Check the fix from the caller's side too.

7.8 Summary

- `'static` is the largest region: it covers the whole run of the program and outlives every other lifetime. It appears in two positions with different meanings.
- `&'static T` is a reference whose referent stays valid until the program ends. It comes from literals, `static` and `const` items, constant promotion (constant values with no destructor and no interior mutability), leaking (`Box::leak`, `String::leak`), and globals initialized at run time (`OnceLock`, `LazyLock`).
- `T: 'static` is a bound on a type: every borrow inside `T` is valid until the program ends. Owning types satisfy it, whatever the lifetime of the value; a `'static` value can be dropped a line after it is created. The test is "does the type hold a borrow that can expire?".
- APIs ask for `T: 'static` when they keep a value for a time they cannot name: `thread::spawn` (E0373 for a closure that borrows a local, E0521 for a borrow that came in as a parameter), `Any` and `TypeId` (lifetimes are erased, so downcasting could forge them), stored trait objects (`Box<dyn Trait>` means `Box<dyn Trait + 'static>`), and generic code that calls any of these (E0310: write `T: 'static` on your parameter).
- `thread::scope` accepts borrows because every thread it spawns is joined before it returns.
- When the compiler asks for `'static`, find the API that asks, then own, scope, mix or, for process-lifetime data only, leak. Writing `&'static` on a parameter moves the error to the callers.

7.9 Exercises

1. Which of these calls compile, given `fn requires_static<T: 'static>(_value: T) {}`? Check your answers by adding them to a copy of `ch07_static_type_dropped.rs`.

```
requires_static(&42);
requires_static(&[1, 2, 3]);
let n = 42;
requires_static(&n);
requires_static(&String::new());
```

Hint: two of them are references to promoted constants. A local variable is never promoted, and neither is a value with a destructor (the last call is E0716, "temporary value dropped while borrowed", which Chapter 5 explains).

2. Rewrite `count_in_background` from `ch07_spawn_param.rs` so that it compiles without changing its signature and without cloning the strings. *Hint:* the thread only needs to run while the function runs; `thread::scope` expresses exactly that, and its result can be returned directly.
3. Change the `Registry` of `ch07_callbacks.rs` so that it accepts closures that borrow local variables, then register a closure that borrows `prefix` without `move`. What can a `Registry<'a>` no longer do that the original could? *Hint:* the field becomes `Vec<Box<dyn Fn(&str) -> String + 'a>>`. Try to return the registry from the function that owns `prefix`, or to store it in a `static`.
4. Write `fn spawn_all<T: Send>(items: Vec<T>) -> Vec<thread::JoinHandle<()>>`, which spawns one thread per item and drops the item there. Fix the error, then decide whether `spawn_all(vec!["a", "b"])` and `spawn_all(vec![name.as_str()])` (with a local `name: String`) compile. *Hint:* E0310 again. After the fix, `Vec<&'static str>` satisfies the bound; a vector of borrows of a local does not.
5. Write an interner `fn intern(name: &str) -> &'static str` that leaks each distinct name at most once, so that the memory it uses is bounded by the number of distinct names, not by the number of calls. *Hint:* keep the leaked names in a `static NAMES: OnceLock<Mutex<HashSet<&'static str>>>`; look the name up first, and leak (`name.to_owned().leak()`) only when it is missing.